

lab05

October 30, 2025

1 Primeiro Experimento

Para realização da primeira atividade, seguindo o Teorema de Nyquist, foi adotada uma frequência maior que a frequência mínima (duas vezes a frequência de Nyquist), sendo assim a taxa de amostragem escolhida foi de 250 Hz (aprox. $2 \times (60 \times 2)$). A escolha de uma taxa maior que a frequência mínima de Nyquist foi para que se tornasse melhor a visualização do gráfico do sinal amostrado.

```
[8]: import numpy as np
import matplotlib.pyplot as plt

# ===== PRIMEIRA FIGURA: Taxa de Amostragem 250 Hz
# =====
# Definindo os parâmetros do sinal
amplitude1 = 5 # Amplitude em volts
frequencia1 = 60 # Frequência em Hz
tempo_total1 = 2/frequencia1 # Tempo total de amostragem em segundos

# Gerando o sinal senoidal
tempo1 = np.linspace(0, tempo_total1, 1000) # Vetor de tempo
sinal_senoidal1 = amplitude1 * np.sin(2 * np.pi * frequencia1 * tempo1)

# Amostrando o sinal
taxa_amostragem1 = 250 # Taxa de amostragem em Hz (adequada para Nyquist)
tempo_amostragem1 = 1 / taxa_amostragem1
amostras1 = np.arange(0, tempo_total1, tempo_amostragem1)
sinal_amostrado1 = amplitude1 * np.sin(2 * np.pi * frequencia1 * amostras1)

# Plotando os gráficos
fig1 = plt.figure(figsize=(10, 8))

# Gráfico do sinal senoidal
ax1 = plt.subplot(3, 1, 1)
plt.plot(tempo1, sinal_senoidal1)
plt.title('Sinal Senoidal 1')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)
```

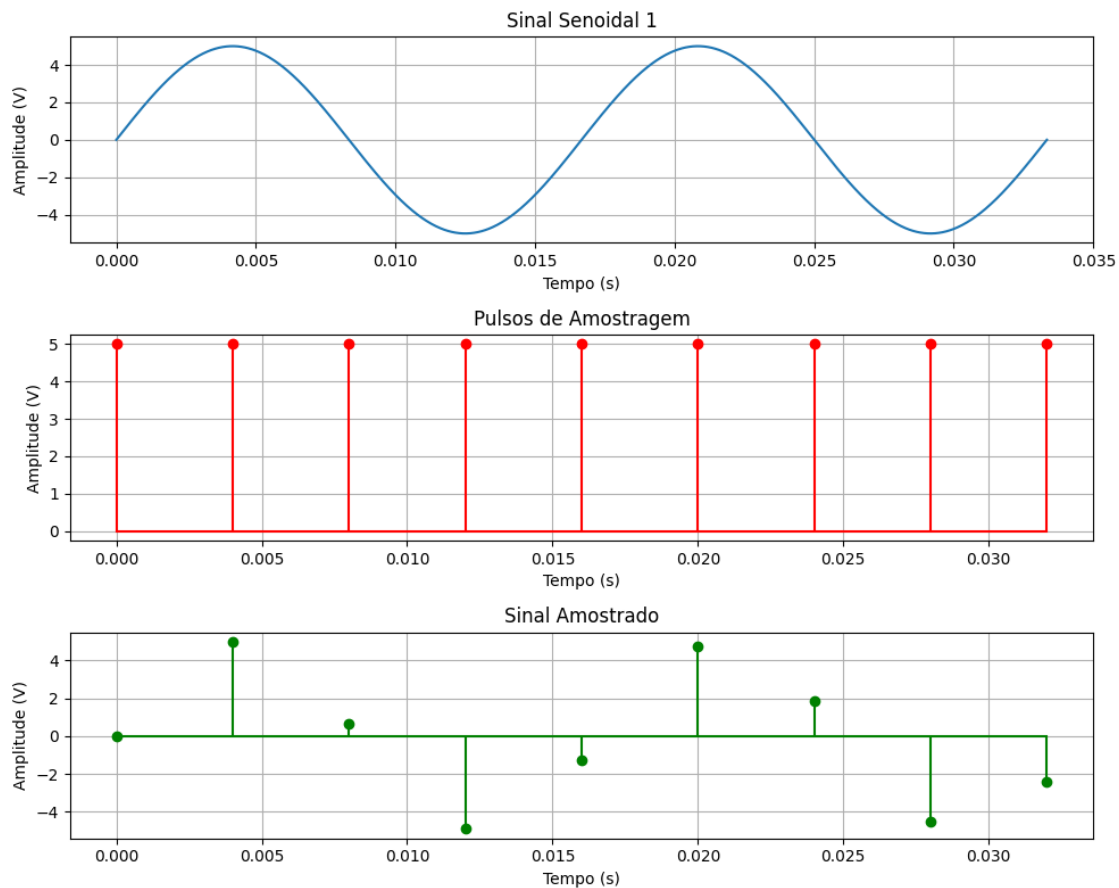
```

# Gráfico dos pulsos de amostragem
ax2 = plt.subplot(3, 1, 2)
plt.stem(amostras1, np.ones(len(amostras1)) * amplitude1, linefmt='r-',
        ↪markerfmt='ro', basefmt='r-')
plt.title('Pulsos de Amostragem')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Gráfico do sinal amostrado
ax3 = plt.subplot(3, 1, 3)
plt.stem(amostras1, sinal_amostrado1, linefmt='g-', markerfmt='go',
        ↪basefmt='g-')
plt.title('Sinal Amostrado')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Ajustando o espaçamento entre os subplots
plt.tight_layout()

```



Com taxa de amostragem escolhida, ao se aplicar a equação 1, tem-se um número de amostras igual a 9. Além disso, pode-se também se obter o tempo de duração de cada amostra, que é igual ao inverso da taxa de amostragem utilizada, sendo assim a duração de cada amostra é de 4 ms. Fazendo outro teste, desta vez com 1200Hz de taxa de amostragem, o resultado foi o seguinte

```
[9]: # ===== SEGUNDA FIGURA: Taxa de Amostragem 1200 Hz
# =====
# Amostrando o sinal
taxa_amostragem1 = 1200 # Taxa de amostragem em Hz
tempo_amostragem1 = 1 / taxa_amostragem1
amostras1 = np.arange(0, tempo_total1, tempo_amostragem1)
sinal_amostrado1 = amplitude1 * np.sin(2 * np.pi * frequencia1 * amostras1)

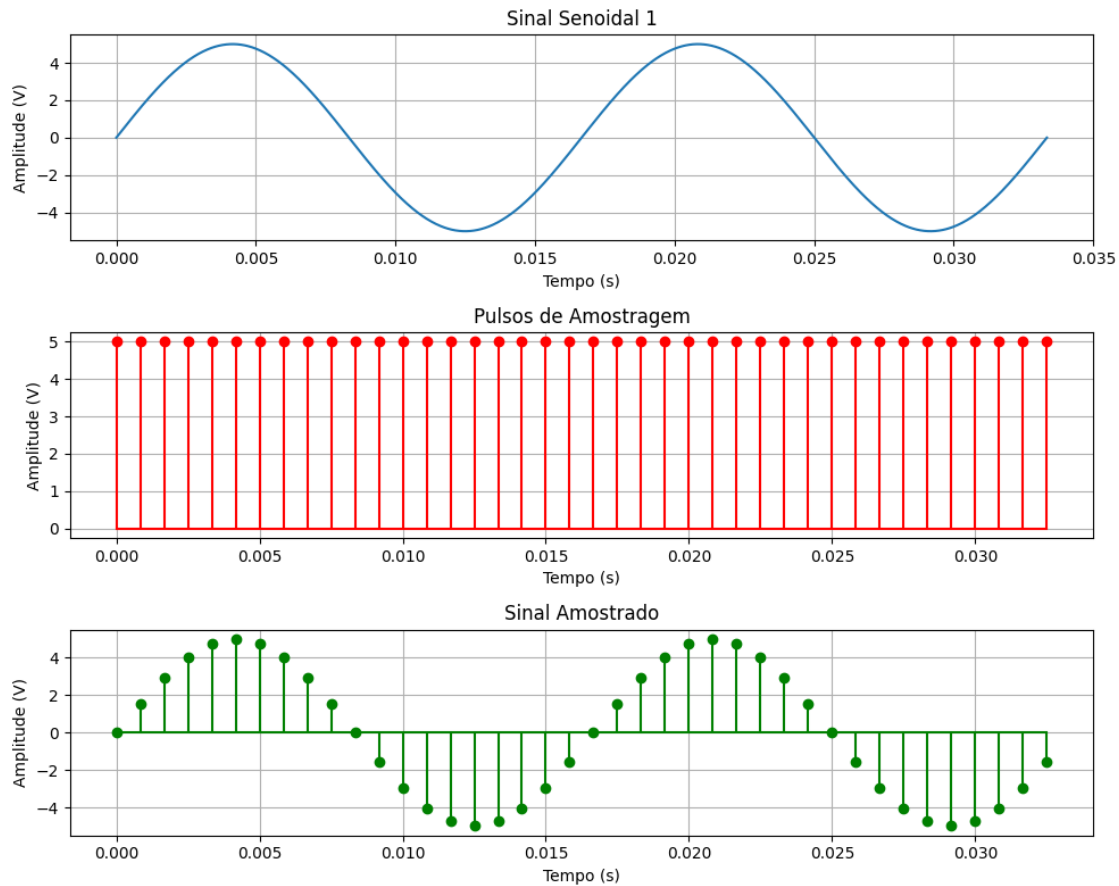
# Plotando os gráficos
fig2 = plt.figure(figsize=(10, 8))

# Gráfico do sinal senoidal
ax1 = plt.subplot(3, 1, 1)
plt.plot(tempo1, sinal_senoidal1)
plt.title('Sinal Senoidal 1')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Gráfico dos pulsos de amostragem
ax2 = plt.subplot(3, 1, 2)
plt.stem(amostras1, np.ones(len(amostras1)) * amplitude1, linefmt='r-',
        markerfmt='ro', basefmt='r-')
plt.title('Pulsos de Amostragem')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Gráfico do sinal amostrado
ax3 = plt.subplot(3, 1, 3)
plt.stem(amostras1, sinal_amostrado1, linefmt='g-', markerfmt='go',
        basefmt='g-')
plt.title('Sinal Amostrado')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

plt.tight_layout()
```



Este resultado foi bem mais satisfatório que o primeiro, considerando que houve mais precisão na captação do sinal através dos pulsos. Seu tempo de amostragem foi de 0,83ms

2 Segundo experimento

Na segunda atividade, como foi pedido para que se utilizassem 20 amostras, foi calculado, com o uso da equação 1, que a taxa de amostragem utilizada deveria ser de 600 Hz, tendo assim um tempo de amostragem de 1,7 s, aproximadamente

```
[12]: # ===== TERCEIRA FIGURA: Taxa de Amostragem 600 Hz
# =====
# Definindo os parâmetros do sinal
amplitude = 5 # Amplitude em volts
frequencia = 60 # Frequência em Hz
tempo_total = 2/frequencia # Tempo total de amostragem em segundos

# Gerando o sinal senoidal
tempo = np.linspace(0, tempo_total, 1000) # Vetor de tempo
sinal_senoidal = amplitude * np.sin(2 * np.pi * frequencia * tempo)
```

```

# Amostrando o sinal
taxa_amostragem = 600 # Taxa de amostragem em Hz
tempo_amostragem = 1 / taxa_amostragem
amostras = np.arange(0, tempo_total, tempo_amostragem)
sinal_amostrado = amplitude * np.sin(2 * np.pi * frequencia * amostras)

# Plotando os gráficos
fig3 = plt.figure(figsize=(10, 8))

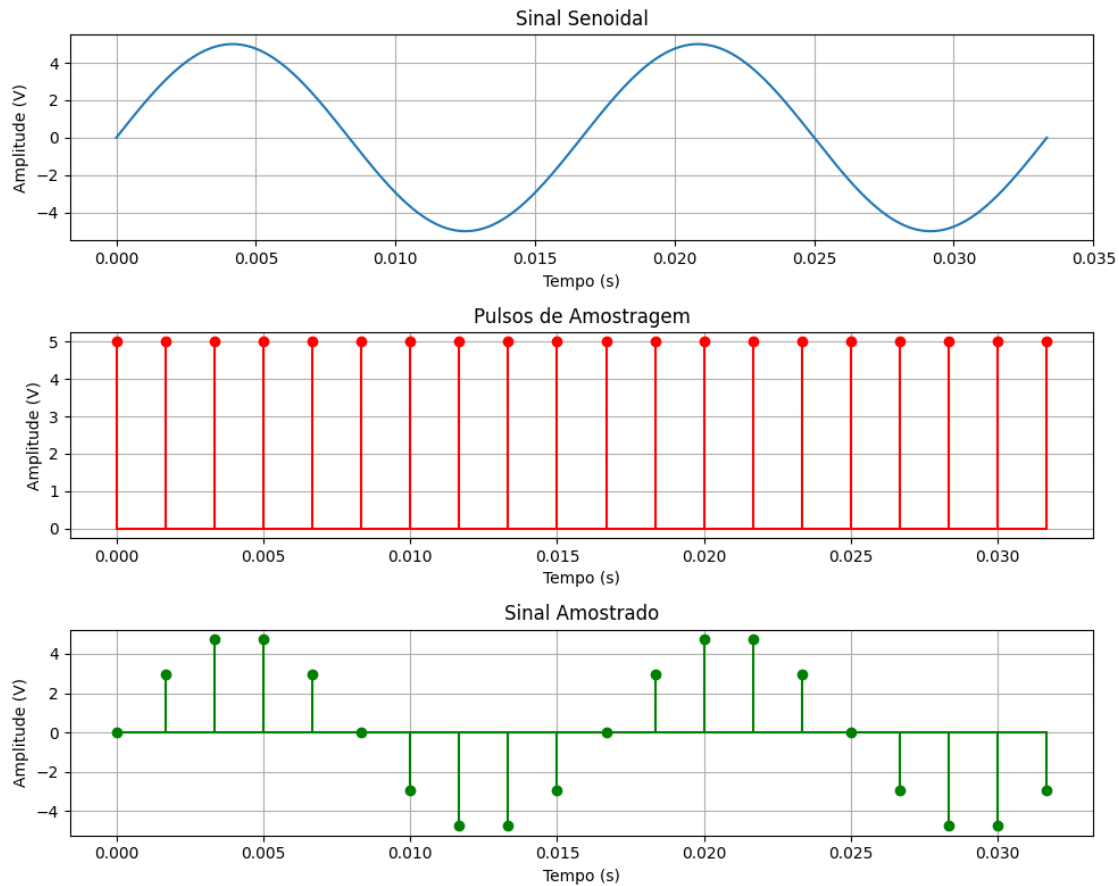
# Gráfico do sinal senoidal
ax1 = plt.subplot(3, 1, 1)
plt.plot(tempo, sinal_senoidal)
plt.title('Sinal Senoidal')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Gráfico dos pulsos de amostragem
ax2 = plt.subplot(3, 1, 2)
plt.stem(amostras, np.ones(len(amostras)) * amplitude, linefmt='r-',
        ↪markerfmt='ro', basefmt='r-')
plt.title('Pulsos de Amostragem')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

# Gráfico do sinal amostrado
ax3 = plt.subplot(3, 1, 3)
plt.stem(amostras, sinal_amostrado, linefmt='g-', markerfmt='go', basefmt='g-')
plt.title('Sinal Amostrado')
plt.xlabel('Tempo (s)')
plt.ylabel('Amplitude (V)')
plt.grid(True)

plt.tight_layout()

```



Assim como é possível de se observar, o uso da taxa calculada resultou realmente em 20 pulsos. Devido ao maior número de amostras, em comparação com a primeira figura, pode ser visto que o sinal amostrado apresenta melhor o comportamento da senoide que o primeiro teste, mas pior que o de 1200Hz que fornece mais informações sobre suas curvas. Para a realização da terceira e última atividade, o sinal escolhido para ser quantificado foi o da atividade 2

```
[13]: # ===== QUARTA FIGURA: Quantização com Diferentes Bits
# =====
# Definindo o sinal amostrado e outras variáveis necessárias
amplitude = np.max(np.abs(sinal_amostrado)) # Determinando a amplitude máxima
amostras_indices = np.arange(len(sinal_amostrado)) # Vetor de amostras

# Definindo os diferentes números de bits para quantização
num_bits_list = [1, 3]

# Criando uma nova figura para os subplots
fig4 = plt.figure(figsize=(10, 8))

# Loop para plotar os gráficos com diferentes números de bits
```

```

for i, num_bits in enumerate(num_bits_list):
    nivel_quantizacao = 2**num_bits

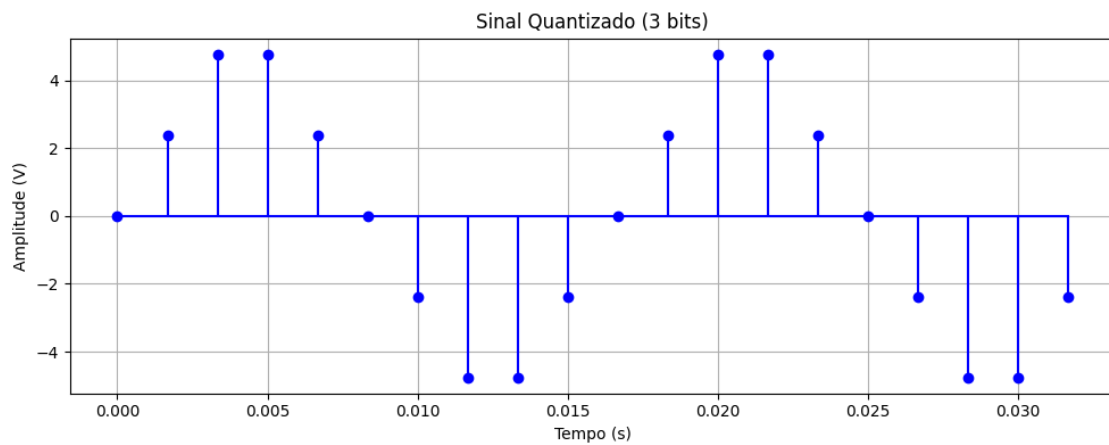
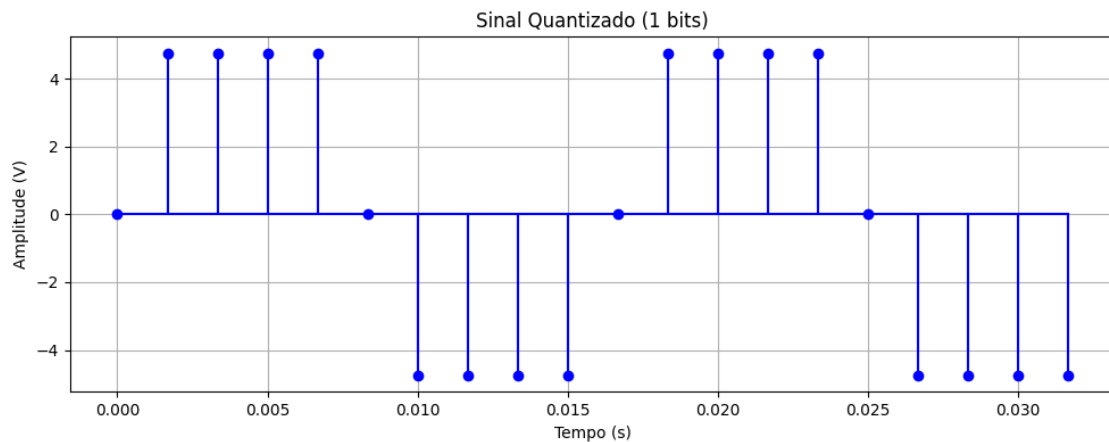
    # Quantização do sinal amostrado
    sinal_quantizado = np.round(sinal_amostrado * (nivel_quantizacao /
    ↳(2*amplitude))) * (2*amplitude / nivel_quantizacao)

    # Criando um subplot para cada sinal quantizado
    plt.subplot(len(num_bits_list), 1, i+1)
    plt.stem(amostras, sinal_quantizado, linefmt='b-', markerfmt='bo',
    ↳basefmt='b-')
    plt.title(f'Sinal Quantizado ({num_bits} bits)')
    plt.xlabel('Tempo (s)')
    plt.ylabel('Amplitude (V)')
    plt.grid(True)

plt.tight_layout()

# Mostrando todas as figuras
plt.show()

```



Foi realizado o teste com 1 e 3 bits (2 e 8 níveis). No primeiro teste obtive uma onda quadrada, pois com 1 bit só é possível ter dois níveis (0 e 1), não sendo possível captar as informações da senoide. No teste com 3 bits, o resultado já foi bem mais satisfatório, podendo retirar algumas informações mas com perdas. Ao aumentar o número de bits para mais que 3, os resultados não são significantes, uma vez que o número de amostras é baixo. Caso o número de amostras aumentasse, seria recomendável aumentar o número de bits (e consequentemente de níveis)